

Lecture 1 Introduction to Algorithm

Definition of algorithms

A sequence of steps which is used to solve a category of problems

- Unambiguous
- Mechanical
- Finite
- Input/output

Algorithm representation

- Program
- Pseudocode
 - `for ( to for increment and downto for decrement; by for changes greater than 1), while , if-else , repeat-until`
 - `//` indicates the comment in a line
 - Not use global variables without explicit indication
 - `A[i]` indicates the `i` th element of array `A` .
 - Access a particular attribute by the object name followed by a dot and then followed by the attribute name. `A.length`
 - `return` transfer control to calling procedure
 - `error` indicates an error occurred.
- Flow chart

Algorithm vs. Program

- Program could be a way to express algorithms
- Algorithms are for people to communicate while programs are for machines to run
- Algorithms are abstract while program is precise and concrete.

Asymptotic Notation

Use basic steps to express running time

- too precise
- depend on particular machines

- Leave out lower-order terms and the coefficient in the leading term(e.g.,
 $5n^3 + 4n + 3 = O(n^3)$)

O notation

$$f = O(g)$$

means f grows no faster than g if **there is** a constant $c > 0$ such that $f(n) \leq c \cdot g(n)$

When prove, we prove that

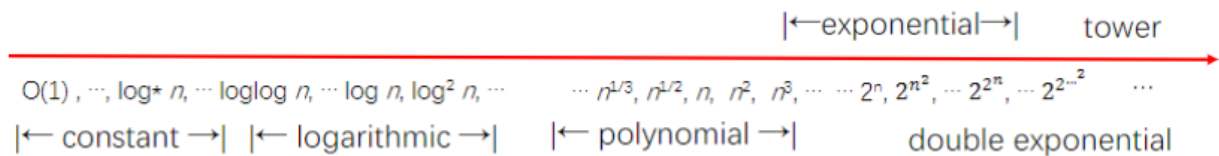
$$\frac{f(n)}{g(n)} \leq c$$

The following are proved similarly.

Extended Definitions

- $f(n) = o(g(n))$: for **any** constant c , $f(n) < c \cdot g(n)$ when n is sufficiently large
- $f(n) = \Omega(g(n))$: for **some** constant c , $f(n) \geq c \cdot g(n)$ when n is sufficiently large
- $f(n) = \omega(g(n))$: for **any** constant c and large n , $f(n) > c \cdot g(n)$
- $f(n) = \Theta(g(n))$: $f(n) = O(g(n))$, $f(n) = \Omega(g(n))$

Spectrum of functions



where $\log^* n$ is $\log \log \log \log \dots n$

Commonsense rules

- Multiplicative constant can be omitted. ($14n^2$ becomes n^2)
- n^a dominates n^b if $a > b$ (n^2 dominates n)
- Any exponential dominates any polynomials
- Any polynomials dominates any logarithm

Properties of Asymptotic Notations

- Transitivity
 - $f(n) = \Theta(g(n)), g(n) = \Theta(h(n)) \implies f(n) = \Theta(h(n))$
 same to small and large o, small and large omega.
- Reflexivity

- $f(n) = \Theta(f(n))$
same for large O, large Omega
- Symmetry
 - $f(n) = \Theta(g(n)) \Leftrightarrow g(n) = \Theta(f(n))$
- Mutual Symmetry
 - $f(n) = O(g(n)) \Leftrightarrow g(n) = \Omega(f(n))$
same to small o and small omega
- Arithmetic operations
 - $O(f(n)) + O(g(n)) = O(f(n) + g(n)) = O(\max\{f(n), g(n)\})$
 - $O(f(n)) * O(g(n)) = O(f(n) * g(n))$
 - $O(cf(n)) = O(f(n))$
 - $g(n) = O(f(n)) \implies O(f(n)) + O(g(n)) = O(f(n))$